Clemens Höglinger

# Zeitlich robuste Erkennung und Zustandsmodellierung von Katzen und Beutetieren mittels Deep Learning

Bachelorarbeit

zur Erlangung des akademischen Grades

Bachelor of Science in Engineering (BSc)

Bachelorstudiengang
Software Engineering
Fachhochschule Oberösterreich
Hagenberg

Jahr der Einreichung: 2026

Betreuung: Dr. Gerald Adam Zwettler

© Copyright 2026 Clemens Höglinger

Diese Arbeit wird unter den Bedingungen der Creative Commons Lizenz *Attribution-NonCommercial-NoDerivatives 4.0 International* (CC BY-NC-ND 4.0) veröffentlicht – siehe https://creativecommons.org/licenses/by-nc-nd/4.0/.

# Eidesstattliche Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder inhaltlich den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und ist mit dem elektronisch eingereichten Textdokument identisch.

Hagenberg, am 20. August 2026

Clemens Höglinger

# Dokumentation der Nutzung von KI-Tools

Über den Einsatz von Hilfsmitteln der generativen Künstlichen Intelligenz wie Chatbots, Bildgeneratoren, Programmieranwendungen, Paraphrasier- oder Übersetzungstools dokumentiere ich, dass ich Hilfsmittel der generativen Künstlichen Intelligenz verwendet habe,

- um Literatur zu finden.
- um Teile der Arbeit Korrektur zu lesen und den sprachlichen Stil zu verbessern.

# Inhaltsverzeichnis

**Eidesstattliche Erklärung** ii

**Dokumentation der Nutzung von KI-Tools** iii

**Kurzfassung** vi

**Abstract** vii

**1 Einleitung** 1

**2 Theoretische Grundlagen** 2
2.1 Objekterkennung mit YOLO . . . 2
2.2 Mehrfach-Objektverfolgung mit ByteTrack . . . 2
2.3 Bildklassifikation mit YOLO . . . 3
2.4 Zustandsautomaten . . . 3

**3 Material und Versuchsgrundlage** 5
3.1 Datensätze . . . 5
3.2 Hardware und Softwareumgebung . . . 5

**4 Methodik und Entwicklung** 7
4.1 Gesamtarchitektur und Datenfluss . . . 7
4.2 Vorverarbeitung für Infrarot-Nachtsicht . . . 8
4.3 Training und Anpassung des Objektdetektors (YOLO26n) . . . 8
4.4 Automatisierte Generierung des Klassifikations-Datensatzes . . . 8
4.5 Bildausschnitt-Extraktion . . . 9
4.6 Training des Bildklassifikators (YOLO-Klassifikation) . . . 9
4.7 Objektverfolgung und Entwicklung des Zustandsmodells . . . 9

**5 Design und Implementierung** 11
5.1 Softwarearchitektur und Kernkomponenten . . . 11
5.1.1 Das Modul `models.py`: Die `CatFlapPipeline` . . . 11
5.1.2 Das Modul `state_machine.py`: Zeitliche Aggregation . . . 12
5.1.3 Die Orchestrierung (`processor.py`) . . . 12
5.2 Die Verbindung zur Hardware (`catflap.py`) . . . 13
5.3 Ausführungsumgebungen . . . 13
5.3.1 Die lokale Testumgebung (`main.py`) . . . 13

5.3.2 Die Mikrocontroller Testumgebung (`pi_main.py`) . . . . . 14
5.3.3 Die Reale Welt (`pi_server.py`) . . . . . 14

**6 Tests und Analysen** **15**
6.1 Modell-Performance (Training & Validation) . . . . . 15
6.1.1 Leistung der Objekterkennung . . . . . 15
6.1.2 Leistung der Bildklassifikation . . . . . 18
6.2 Evaluationsmethodik und Datensatz . . . . . 21
6.2.1 Definition der System-Metriken . . . . . 22
6.3 Ergebnisse der Evaluation . . . . . 22
6.3.1 Analyse der Systemleistung und Kompromisse . . . . . 22
6.4 Ablationsstudie: Relevanz des Zustandsautomaten . . . . . 23
6.4.1 Ablationsstudie: Relevanz der Randerweiterung . . . . . 24
6.4.2 Ablationsstudie: Kontrastverstärkung (CLAHE) im Klassifikator 25
6.4.3 Ablationsstudie: Einstufige vs zweistufige Klassifikation . . . . . 25

**7 Diskussion** **28**
7.1 Beantwortung der zentralen Forschungsfrage . . . . . 28
7.2 Bewertung des Architektur-Ansatzes . . . . . 28
7.2.1 Stärken der Verarbeitungsketten-Kaskadierung . . . . . 29
7.2.2 Die Rolle der asymmetrischen Randerweiterung . . . . . 29
7.3 Schwächen, Limitationen und Fehlerquellen . . . . . 29
7.4 Lokale Datenverarbeitung . . . . . 30

**8 Zusammenfassung und Ausblick** **31**
8.1 Zusammenfassung der Erkenntnisse . . . . . 31
8.2 Ausblick und Weiterentwicklungsmöglichkeiten . . . . . 32
8.2.1 Kontinuierliches Sammeln schwieriger Negativbeispiele . . . . . 32
8.2.2 Verlust der Tiefeninformationen . . . . . 32
8.2.3 Hardware-Beschleunigung . . . . . 32

**Quellenverzeichnis** **33**
Literatur . . . . . 33

# Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. Im Unterschied zu anderen Kapiteln ist die Kurzfassung (und das Abstract) üblicherweise nicht in Abschnitte und Unterabschnitte gegliedert. Auch Fußnoten sind hier falsch am Platz.

Kurzfassungen werden übrigens häufig – zusammen mit Autor*in und Titel der Arbeit – in Literaturdatenbanken aufgenommen. Es ist daher darauf zu achten, dass die Information in der Kurzfassung für sich *allein* (d. h. ohne weitere Teile der Arbeit) zusammenhängend und abgeschlossen ist. Insbesondere werden an dieser Stelle (wie u. a. auch im *Titel* der Arbeit und im *Abstract*) normalerweise *keine Literaturverweise* verwendet! Falls unbedingt solche benötigt werden – etwa weil die Arbeit eine Weiterentwicklung einer bestimmten, früheren Arbeit darstellt –, dann sind *vollständige* Quellenangaben in der Kurzfassung selbst notwendig, z. B. [Zobel J.: *Writing for Computer Science – The Art of Effective Communication*. Springer-Verlag, Singapur, 1997].

Auch sollte daran gedacht werden, dass bei der Aufnahme in Datenbanken Sonderzeichen oder etwa Aufzählungen mit „Knödellisten“ in der Regel verloren gehen. Dasselbe gilt natürlich auch für das *Abstract*.

Inhaltlich sollte die Kurzfassung *keine* Auflistung der einzelnen Kapitel sein (dafür ist das Einleitungskapitel vorgesehen), sondern dem*der Leser*in einen kompakten, inhaltlichen Überblick über die gesamte Arbeit verschaffen. Der hier verwendete Aufbau ist daher zwangsläufig anders als der in der Einleitung.

# Abstract

This should be a 1-page (maximum) summary of your work in English.

Im englischen „Abstract“ sollte inhaltlich das Gleiche stehen wie in der deutschen Kurzfassung. Versuchen Sie daher, die Kurzfassung präzise umzusetzen, ohne aber dabei Wort für Wort zu übersetzen. Beachten Sie bei der Übersetzung, dass gewisse Redewendungen aus dem Deutschen im Englischen kein Pendant haben oder völlig anders formuliert werden müssen und dass die Satzstellung im Englischen sich (bekanntlich) vom Deutschen stark unterscheidet (mehr dazu in Abschn. **??**). Es empfiehlt sich übrigens – auch bei höchstem Vertrauen in die persönlichen Englischkenntnisse – eine kundige Person für das „proof reading“ zu engagieren.

Die richtige Übersetzung für „Diplomarbeit“ ist übrigens schlicht *thesis*, allenfalls „diploma thesis“, auf keinen Fall aber „diploma work“ oder gar „dissertation“. Für „Masterarbeit“ bzw. „Bachelorarbeit“ sind „master's thesis“ bzw. „bachelor's thesis“ die passenden Übersetzungen.

Übrigens sollte für diesen Abschnitt die *Spracheinstellung* in LaTeX von Deutsch auf Englisch umgeschaltet werden, um die richtige Form der Silbentrennung zu erhalten. Mit der Option `smartquotes` werden dabei auch die Anführungszeichen automatisch an die jeweilige Sprache angepasst (s. dazu die Abschnitte **??** und **??**).

# Kapitel 1

# Einleitung

Freigänger-Katzen bringen von Zeit zu Zeit Beutetiere, wie z.b. Mäuse oder Vögel, mit ins Haus, was für den Besitzer sehr unangenehm sein kann. Diese Beutetiere sind auf Kameraaufnahmen oft schwer erkennbar, da sie klein sind oder verdeckt werden. Wenn man versucht, dieses Problem mit reiner Objekterkennung zu lösen, erweisen sich reine Einzelbilder oft als fehleranfällig [7]. Bewegungsunschärfe oder Verdeckungen führen dazu, dass die Beute nicht mit der benötigten Genauigkeit erkannt werden kann. Eine Momentaufnahme ist hier nicht ausreichend, weshalb eine Analyse über mehrere Einzelbilder erforderlich ist.

Diese Bachelorarbeit beschäftigt sich mit der durch künstliche Intelligenz gestützter Erkennung von Beutetieren im Maul einer Katze. Das Ziel dieses Projektes ist es, eine Alternative zu den herrkömlichen Katzenklappen zu bieten und dem Katzenbesitzer das lästige fangen von Beute im Wohnraum zu ersparen.

Da die Bilderkennung unter anderem das Herzstück dieses Projektes ist, habe ich mich intensiv mit verschiedenen Algorithmen beschäftigt. Im ersten Schritt wird das Objektdetektionsmodell YOLOv11 [8] eingesetzt. YOLOv11 erreicht eine hohe Genauigkeit während es auch auf Endgeräten echtzeitfähig bleibt. Anschließend kommt der ByteTrack [16] Algorithmus zum Einsatz. Dieser Algorithmus verfolgt die erkannten Begrenzungsrahmen über die Zeit und sorgt dafür, dass das System die Katze auch bei kurzzeitigen Verdeckungen nicht aus den Augen verliert. ByteTrack sorgt auch dafür, dass jede Katze eindeutig ist sollten sich mehrere Katzen auf dem Videoeinzelbild befinden. Die erkannten Bereiche werden anschließend durch eine weitere künstliche Intelligenz (YOLO-Klassifikation) ausgewertet, und in zwei Kategorien eingeteilt, Katze mit und ohne Beute. Um die Zuverlässigkeit zu erhöhen und Fehlalarme zu vermeiden, trifft man nicht für jedes Videoeinzelbild eine Entscheidung sondern sammelt Daten über mehrere Videoeinzelbilder hinweg und trifft für diese Daten als gesamtes eine Entscheidung. Dafür wurde ein eigener Zustandsautomat (State Machine) eingebunden. Dieser wertet die Historie der Bilder über ein gleitendes Zeitfenster aus und trifft dann eine Entscheidung, ob die Katze beute im Mund trägt oder nicht. In dieser Arbeit wird evaluiert, inwiefern diese Kombination aus Objekterkennung, Klassifikation, Objektverfolgung und Zustandsautomaten eine bessere Lösung ist als die reine Bewertung durch Einzelbilder.

# Kapitel 2

# Theoretische Grundlagen

In diesem Kapitel werden die theoretischen Grundlagen erläutert, diese bilden dass Fundament dieser Arbeit. Es werden ausschließlich Technologien und Algorithmen erklärt, welche in der finalen Arbeit verwendet werden. Dazu zählen Objekterkennung, Objektverfolgung, Bildklassifikation und Zustandsautomaten.

## 2.1 Objekterkennung mit YOLO

Die automatische Bildanalyse lässt sich grundsätzlich in verschiedene Problemklassen unterteilen [12]. Während die reine Bildklassifikation lediglich die Frage beantwortet, ob ein bestimmtes Objekt in einem Bild vorhanden ist, geht die Objekterkennung einen Schritt weiter. Sie ermittelt nicht nur die Klasse des Objekts, sondern auch die genaue Position an der sich das Objekt befindet.

In dieser Arbeit wird YOLO [8] in der Version 26 verwendet. Im Gegensatz zu zweistufigen Detektoren, welche zuerst interessante Bildregionen extrahieren und erst im zweiten Schritt klassifizieren, ist YOLO ein einstufiger Detektor, welcher das gesamte Bild in einem Durchlauf mithilfe eines Faltungsnetzwerk [10] analysiert. Das Bild wird in Raster unterteilt, jede Rasterzelle gibt das Netzwerk mehrere Begrenzungsrahmen und die zugehörigen Klassenwahrscheinlichkeiten und Konfidenzwerte voraus. Der Begrenzungsrahmen ist der Rahmen in dem sich ein Objekt befindet. Die Klassenwahrscheinlichkeit gibt an wie wahrscheinlich das erkannte Objekt zur Klasse Katze gehört. Der Konfidenzwert hingegen gibt an, wie sicher sich das Netzwerk ist, dass es sich überhaupt um ein Objekt handelt [13].

Für diese Arbeit ist YOLO unerlässlich, da ein solches System niedrige Inferrenzzeiten benötigt. Das ist auf der einen Seite wichtig damit das System verlässlich bleibt und die Katze nicht zu lange vor der Klappe warten muss. Das Modell liefert für jede erkannte Katze die exakten Koordinaten dessen Gesichtes, was die Basis für die Objektverfolgung bildet.

## 2.2 Mehrfach-Objektverfolgung mit ByteTrack

Eine Objekterkennung wie YOLO alleine bewertet jedes Videoeinzelbild isoliert. Wenn sich eine Katze auf einem Video befindet, dann wird jedes Einzelbild des Videos separat

betrachtet. Jede erkannte Katze ist also eine neue Katze. Durch eine Objektverfolgung wird es möglich, eine Katze über ein ganzes Video eindeutig zuzuordnen. Eine Mehrfach-Objektverfolgung ist dann die Erweiterung auf mehrere Katzen pro Video. Das ist zwingend erforderlich um eine zeitliche Analyse einer bestimmten Katze durchführen zu können.

In dieser Arbeit erzeugt ein Objektdetektor (in diesem Fall YOLO26) die Begrenzungsrahmen, diese enthalten die genauen Koordinaten der Objekte, in unserem Fall vom Gesicht der Katze. Im zweiten Schritt übernimmt der Tracker. Er versucht die neuen Erkennungen den bekannten zuzuordnen, sollte das nicht möglich sein wird eine neue Identität erzeugt. Im Fall von ByteTrack [16] passiert dieses zuordnen durch Bewegungsvorhersagen welche der Kalman-Filter [9] trifft.

ByteTrack hebt sich von klassischen Trackern ab indem er auch Erkennungen mit niedrigem Konfidenzwert verwendet um Identitätswechsel vorzubeugen, damit dies nicht zu Fehlalarmen führt, ordnet er im ersten Schritt die Begrenzungsrahmen mit hohen Konfidenzwerten zu. Anschließend werden die Objekte mit den niedrigen Konfidenzwerten verglichen welche im vorigen Schritt nicht zugeordnet werden konnten. Dadurch wird die Verfolgung robuster bei Verdeckungen. Daher eignet sich ByteTrack ideal für unvorhersehbare Bewegungen und Okklusionen beim Anwendungsfall einer Katzenklappe.

## 2.3 Bildklassifikation mit YOLO

Die Klassifikation arbeitet wie die Objekterkennung nach dem Prinzip von Faltungsnetzwerken [10]. Moderne YOLO-Versionen, wie in unserem Fall YOLO26, bieten eben nicht nur die Objekterkennung, sondern auch die Klassifikation an. Diese Netzwerke sind durch ihre Architektur für schnelle Inferenz bei hoher Genauigkeit ausgelegt.

Während die Objekterkennung uns zeigt wo sich eine Katze befindet, wissen wir aber noch nicht ob eine Katze Beute im Mund trägt oder nicht. Hier kommt die Bildklassifikation zum Einsatz. Dieser Ansatz hat den Vorteil dass die Klassifikation nicht mehr das ganze Bild verstehen muss, das Modell bekommt lediglich das Gesicht der Katze, dieser Ansatz benötigt einen wesentlich kleineren Datensatz, da die Klassifikation immer eine ähnliche Aufgabe mit wenig Varianz bekommt.

Die Aufgabe des Klassifikators ist somit eine binäre Entscheidung, trägt die Katze Beute im Mund oder nicht. Die Ausgabe des Modells ist ein Konfidenzwert wie wahrscheinlich die Katze Beute trägt.

## 2.4 Zustandsautomaten

Während der Objektdetektor und die Klassifikation Momentaufnahmen liefern, welche durchaus schwanken können, hilft uns bei der Entscheidung ein Zustandsautomat. Er aggregiert die Ergebnisse von Detektor und Klassifikation zeitlich, dadurch wird das Ergebnis nicht stark durch Ausreißer beeinflusst, und wird robuster gegen Fehler.

Ein endlicher Zustandsautomat ist ein Modell, welches eine endliche Menge an Zuständen beinhaltet. Das Modell befindet sich zu einem Zeitpunkt immer in genau einem Zustand. Durch bestimmte Ereignisse oder erfüllen bestimmter Bedingungen, wird der

aktuelle Zustand des Automaten bestimmt, z.B. wenn das System die letzten 5 mal Katze mit Maus erkannt hat, dann wechselt der Zustandsautomat auf Zugang verweigert. Der Übergang von einem Zustand löst für gewöhnlich auch Aktionen aus, in unserem Fall würden wir die Katzenklappe öffnen oder schließen.

Ein einfaches Beispiel für einen Zustandsautomaten ist eine Ampel, das Modell repräsentiert ob die Ampel Rot, Gelb oder Grün ist. Bedingung für das Wechseln ist ein Timer. Die Aktion wäre das Umschalten der Ampel.

In unserem Kontext dient der Zustandsautomat Ungenauigkeiten der Objekterkennung und Klassifikation zu glätten. Anstatt bei einem Ausreißer sofort den Zustand der Katzenklappe zu ändern, wird der Zustandsübergang an Bedingungen (z.b. einen Schwellenwert über ein Zeitfenster) gebunden.

# Kapitel 3

# Material und Versuchsgrundlage

In diesem Kapitel werden die konkreten Ressourcen und Technologien erläutert, welche für das Training der Modelle und die Evaluation der Verarbeitungskette verwendet wurden.

## 3.1 Datensätze

Für die Objekterkennung und Bildklassifikation wurden unterschiedliche Datensätze benötigt, da beide ein spezifisches Problem lösen.

**Datensatz für die Objekterkennung (Katzengesichter):**
Für das Training der Objekterkennung mit YOLO26 wurde ein öffentlicher Kaggle-Datensatz verwendet („Cat Dataset“ von Crawford [4]). In diesem Datensatz ist bereits für jede Katze gekennzeichnet wo sich dessen Gesicht im Bild befindet, außerdem enthält er eine große Variation an Katzen in verschiedensten Lichtverhältnissen, Posen und Hintergründen. Die Daten mussten nur noch leicht umgeändert werden damit sie für das YOLO-Training geeignet waren.

**Datensatz für die Klassifikation (Beutetiere):**
Im Gegensatz zur Objekterkennung wo es bereits einen fertigen Datensatz gab, fand ich zu diesem Problem kaum Daten. Entweder waren Datensätze quantitativ oder qualitativ zu niederwertig. Um dieses Problem zu lösen wurden die Trainingsdaten mit hilfe von KI-generiert. Dadurch konnten gezielt verschiedene Szenarien, sprich unterschiedliche Beutetiere und Posen, erzeugt werden. Durch diese Art und Weise konnte das YOLO26 Modell mit ausreichender Genauigkeit trainiert werden.

## 3.2 Hardware und Softwareumgebung

Das Training der Modelle benötigt im Vergleich zur reinen Verwendung sehr ressourcenintensiv, daher habe ich das Training ausgelagert um schnell an Ergebnisse zu kommen.

**Hardware-Setup:**
Das Training der Modell wurde auf der Plattform Kaggle durchgeführt, diese stellt eine ausreichend starke GPU zur Verfügung welche für das Training unerlässlich war. Die späteren Tests wurden zuerst auf meiner lokalen Systemarchitektur und anschließend noch auf einem Mikrocontroller, in diesem Fall auf dem Raspberry Pi 5, getestet.

**Softwareumgebung:**
Die Implementierung des Systems erfolgte mit Hilfe der Programmiersprache Python [15]. Für Detektor und Klassifikator wurde das `ultralytics`-Paket verwendet. Bilderverarbeitungsoperationen wie z.b. Extrahieren von Bildausschnitten, wurden mit OpenCV [3] realisiert. Für die optimale Inferrenzgeschwindigkeit wurden die Modelle in das TensorFlow Lite [1] Format exportiert, da diese extra für ARM [2] Architekturen optimiert sind, welche von unserem Mikrocontroller verwendet wird.

# Kapitel 4

# Methodik und Entwicklung

In diesem Kapitel wird die konzeptionelle Entwicklung und technische Umsetzung des vorgestellten Erkennungssystems dieser Arbeit erläutert. Es wird der Weg von der Problemstellung bis hin zur Lösung beschrieben. Dabei wird genauer auf die Designentscheidungen und die Architektur der einzelnen Komponenten der Verarbeitungskette eingegangen.

## 4.1 Gesamtarchitektur und Datenfluss

Um zuverlässig beurteilen zu können ob eine Katze ein Beutetier im Maul trägt, reicht weder eine reine Objekterkennung noch eine reine Bildklassifikation aus. Da wir nur einen begrenzten Datensatz an Katzen mit Beutetieren zur Verfügung haben, ist es unerlässlich diese Aufgabe auf mehrere Stufen zu teilen. Daraus ergibt sich eine Verarbeitungskette aus Objekterkennung, Objektverfolgung, Bildklassifikation und zeitlicher Zustandsmodellierung. Dabei sieht die finale Verarbeitungskette wie folgt aus:

1. Ein Einzelbild der Kamera wird aufgenommen.
2. Die YOLO Objekterkennung analysiert das Bild.
   (a) Wenn keine Katze gefunden wurde, endet die Kette hier.
   (b) Wird mindestens eine Katze gefunden, so wird jedes Katzengesicht aus dem Bild extrahiert.
3. Der ByteTrack-Algorithmus ordnet den Katzengesichtern eine Identität zu oder erstellt eine neue, sollte die Katze frisch ins Bild gekommen sein.
4. Das YOLO-System untersucht jedes Katzengesicht einzeln. Danach meldet es für jede Katze entweder ‚sauber‘ oder ‚Beute‘.
5. Der Zustandsautomat aggregiert die Ergebnisse der Bildklassifikation über die Zeit und trifft dann eine finale Entscheidung, in welchem Zustand (z. B. „Katze mit Beute“) sich die verfolgten Objekte befindet.
   (a) Trägt eine Katze Beute im Maul so wird die Klappe geschlossen.
   (b) Sind alle Katzen sauber so wird die Klappe geöffnet.

## 4.2 Vorverarbeitung für Infrarot-Nachtsicht

Da Katzen ihre Beute nicht nur am Tag bei optimaler Belichtung ins Haus bringen, ist eine zentrale Anforderung an das System das es auch Nachts und bei schlechter Belichtung funktioniert. Das bringt eine weitere Schwierigkeit mit sich: Nachtsichtkameras schalten bei Dunkelheit auf Infrarot um, wodurch Graustufenbilder entstehen.

Damit das neuronale Netz die Farbinformationen nicht erlernt, werden alle Bilder vor dem Training und der Inferenz zu Graustufenbildern konvertiert. Andernfalls würde weder Objekterkennung noch Bildklassifikation bei Nacht zuverlässig funktionieren.

Zusätzlich kommt ein Algorithmus namens CLAHE (Contrast Limited Adaptive Histogram Equalization) zum Einsatz, der in [17] beschrieben wird. Vor allem bei schlechten Lichtverhältnissen oder dunklen Katzen, verbessert CLAHE den lokalen Kontrast. Dadurch gelingt es, Katzengesichter und Beute in der Nacht besser zu erkennen. Er löst vor allem das Problem des schwachen Kontrasts bei Katzen mit dunkler Fellfarbe.

## 4.3 Training und Anpassung des Objektdetektors (YOLO26n)

Für die Objekterkennung kommt die Archtitektur YOLO26 in der Ausführung `yolov11n` zum Einsatz. Sie ist die kleinste und bietet hohe Erkennungsgenauigkeit während es die notwendige Inferenzgeschwindigkeit für Mikrocontroller liefert.

Standardmodelle von YOLO26 sind nicht spezifisch auf die Erkennung von Katzengesichtern abgestimmt. Daher wurde eine Feinabstimmung für genau dieses Problem durchgeführt. Grundlage hierfür ist der Kaggle-Datensatz (Cat Dataset von Crawford [4]), da dieser bereits Annotationen für das Gesicht enthält. Der Datensatz beinhaltet die Koordinaten für Augen, Ohren und Mund. Um diese für YOLO nutzbar zu machen, müssen sie in ein Begrenzungsrahmen-Format umgewandelt werden. Dafür wurde ein Algorithmus entwickelt, der die Breite des Rahmens durch die Ohren und die Höhe durch die Ohren und den Mund vorgibt.

Die Feinabstimmung des YOLO26n-Modells erfolgte über 100 Epochen und einer Bildauflösung von 640x640 Pixeln und der Standard YOLO-Augmentation [14]. Um eine Überanpassung des Modells zu vermeiden, wird das Training abgebrochen, sollte in den letzten zehn Epochen keine Verbesserung mehr eintreten. [6].

## 4.4 Automatisierte Generierung des Klassifikations-Datensatzes

Für die Klassifikation, ob die Katze Beute im Maul trägt, gab es leider keinen brauchbaren Datensatz. Für das Training werden zwei Arten von Bildern benötigt, Katzen mit und Katzen ohne Beute. Die Katzen ohne Beute können wir wieder aus dem Kaggle-Datensatz (Cat Dataset von Crawford [4]) verwenden. Katzen mit Beute auf der anderen Seite sind extrem schwierig zu bekommen, entweder sie sind qualitativ minderwertig oder Urheberrechtlich geschützt.

Die Bilder von Katzen mit Beute, lies ich mir daher von Gemini [5] generieren. Mithilfe der offiziellen Gemini Application Programming Interface (API), entwickelte ich einen Algorithmus, der unter Berücksichtigung des Benutzungslimits automatisch Bilder von Katzen mit Beute in unterschiedlichen Posen und Entfernungen generiert.

Für die Vorbereitung des Datensatzes zur Feinabstimmung des YOLOv11n Klassifikators, wird der zuvor trainierte Katzengesichts-Detektor verwendet. Er erkennt und extrahiert die Katzengesichter und schlüsselt diese dann direkt auf zwei Gruppen auf, Katzen mit und ohne Beute. Dadurch sind die Bilder fertig annotiert und benötigen keine weitere Aufmerksamkeit mehr.

Der Klassifikator hat somit immer die gleiche Eingangslage: Er bekommt stets ein Katzengesicht ohne viel Hintergrund. Das bildet die perfekte Trainingsgrundlage, da die Klassifikation in der späteren Inferenz genau die gleichen Eingangsbilder erhält.

## 4.5 Bildausschnitt-Extraktion

Da der Eingang des Klassifikators der Ausgang des Detektors ist, müssen beide die erforderlichen Informationen erhalten. Während der Detektor am besten funktioniert, wenn er nur auf das Katzengesicht trainiert wird, benötigt er auch die Information, was sich unter dem Maul des Katzengesichts befindet.

Daher wird der Begrenzungsrahmen des Detektors verändert bevor ihn der Klassifikator bekommt. Das Katzengesicht wird nach untenhin um 30 Pixel erweitert um sicher zu gehen das die Beute im Bild ist.

Das extrahierte Katzenbild wird danach durch CLAHE im Kontrast verstärkt. Dadurch das die Eingabeschicht des Klassifikators ein 3-Kanal-Bild erwartet wird die Graustufe dreifach dupliziert (`cv2.cvtColor(gray_crop, cv2.COLOR_GRAY2BGR)`).

## 4.6 Training des Bildklassifikators (YOLO-Klassifikation)

Als Bildklassifikator kommt wie beim Detektor ein YOLO-Modell zum Einsatz. Dadurch das wir uns bereits in der Ultralytics Umgebung befinden und moderne YOLO-Versionen hochoptimierte Netzwerke für Klassifikationsaufgaben bereitstellen, war diese Entscheidung naheliegend.

Das Modell bekommt ein 320x320 Pixel großes Bild und liefert einen Konfidenzwert, der angibt mit welcher Wahrscheinlichkeit sich Beute auf dem Bild befindet. Dadurch dass der Datensatz des Modells relativ klein ist, wird hier stark auf Datenvervielfältigung gesetzt (zufällige Rotation des Bildes, horizontale Spiegelung). Durch diese Augmentationsschritte wird das Modell robuster gegen unbekannte Posen der Katzen.

## 4.7 Objektverfolgung und Entwicklung des Zustandsmodells

Wenn sich eine Katze der Klappe nähert, ändert sich die Qualität der Bilder durch Bewegungsunschärfe oder ungünstige Winkel stark. Das hat zur Folge, dass die Ausgaben des Bildklassifikators schwanken. Wenn z.b. die Beute kurzzeitig verdeckt ist, würde das zur Folge haben das die Katze ins Haus darf.

Damit dieses Problem eingedämpft wird, wurde die Objekterkennung um den ByteTrack-Algorithmus erweitert. Dadurch bekommt eine Katze beim ersten auftreten im Bild eine eindeutige Objekt-ID, diese ID bleibt in den darauffolgenden Bildern für diese Katze gleich. Basierend auf der Eindeutigkeit jeder Katze konnte nun ein Zustandsautomat entwickelt werden. Diese Klasse `StateMachine` verwaltet für jede erkannte Katze

(`track_id`) ein `TrackState`-Objekt. Das Modell kennt drei Zustände:

- `NO_CAT`: Es befindet sich kein relevantes Objekt im Sichtfeld.
- `CAT_NO_PREY`: Eine Katze wurde erkannt, trägt jedoch keine Beute.
- `CAT_WITH_PREY`: Eine Katze wurde erkannt und trägt mit hinreichender Sicherheit eine Beute.

Damit nun die Systementscheidung nicht durch das aktuellste Einzelbild getroffen wird, speichert das `TrackState`-Objekt eine Historie der Konfidenzwerte des Klassifikators der letzten 15 Bilder. Der aktuelle Zustand wird bei jedem Bild neu berechnet, dabei wird einfach der gleitende Durchschnitt der Historie berechnet. Wird der Schwellenwert von 0,8 überschritten, so wird die Katze als `CAT_WITH_PREY` markiert. Fällt der Wert darunter wird sie als `CAT_NO_PREY` markiert.

Der Schwellenwert von 0,8 wurde bewusst so hoch gewählt, um zu vermeiden, dass die Katze zu Unrecht draußen bleiben muss. Dieser Wert ist aber nicht in Stein gemeißelt. Wenn es für den Endanwender wichtiger wäre, Beute im Haus um jeden Preis zu vermeiden würde man den Wert niedriger ansetzen, beispielsweise bei 0,3.

Um Speicherlecks in der Zustandsmaschine zu verhindern, werden Katzen die das Bildfeld verlassen haben oder die länger als 30 Bilder nicht mehr gefunden werden (`max_missed_frames`) aus dem Speicher entfernt.

# Kapitel 5

# Design und Implementierung

Nachdem sich das vorherige Kapitel mit der Methodik beschäftigt hat, geht dieses Kapitel auf die konkrete Realisierung des Systems ein. Dabei wird erläutert, wie die Architektur aufgebaut ist und wie die einzelnen Softwarekomponenten ineinandergreifen. Darüber hinaus wird auf die verwendeten Datenstrukturen eingegangen.

## 5.1 Softwarearchitektur und Kernkomponenten

Die Implementierung erfolgte in der Programmiersprache Python und hält sich streng an die Regeln der objektorientierten Programmierung, damit das System wart- und erweiterbar bleibt. Dabei wurde das Kernsystem in drei wesentliche Skripte unterteilt: Erkennungsmodelle (`models.py`), Zustandsautomat (`state_machine.py`) und die Orchestrierung (`processor.py`).

### 5.1.1 Das Modul `models.py`: Die `CatFlapPipeline`

Die Klasse `CatFlapPipeline` verwaltet Detektor und Klassifikator und vereinfacht deren Verwendung. Beim Initialisieren des Objekts werden beide Modelle geladen. Dabei werden die Pfade (absolut oder relativ) zu den Modellen ergänzt. Zudem wird angegeben, für welches Modell der CLAHE-Algorithmus angewendet werden soll.

Die Methode `run_detector` konvertiert das Bild in Graustufen und verwendet optional den CLAHE-Algorithmus. Anschließend wird die Objekterkennung aufgerufen, durch den Parameter `persist=True` wird der in Ultralytics integrierte ByteTrack-Tracker aktiviert, welcher uns die Begrenzungsrahmen samt Objekt-IDs liefert.

Die Methode `run_classifier` schneidet das Bild wie in Kapitel 4.5 beschrieben zusammen. Im folgenden Codeausschnitt ist zu sehen, wie dieses Zuschneiden implementiert ist und welche Auswirkungen es auf das Katzengesicht hat. Durch das Erweitern nach unten stellen wir sicher, dass das Modell genügend Informationen über mögliche Beute im Maul der Katze erhält.

**Listing 5.1:** Asymmetrische Randerweiterung für den Bildausschnitt

```
1 x1, y1, x2, y2 = map(int, box)
2 # Add padding (heavy on bottom for dangling prey)
3 pad_w = 0
```

```
pad_top = 0
pad_bottom = 30
x1 -= pad_w
y1 -= pad_top
x2 += pad_w
y2 += pad_bottom
```

Im nächsten Schritt wird wieder optional der CLAHE-Algorithmus angewendet. Anschließend wird der Bildausschnitt des Katzengesichtes vom Bildklassifikator verarbeitet, dieser liefert uns dann den Konfidenzwert für die Klasse `CAT_WITH_PREY`.

### 5.1.2 Das Modul `state_machine.py`: Zeitliche Aggregation

Der Zustandsautomat ist durch die Klasse `StateMachine` abgebildet. Der Status einer einzelnen Katze wird in der Klasse `TrackState` gespeichert, diese speichert die Historie der Konfidenzwerte in einer `collections.deque` mit einer fixen Maximallänge (`history_length = 15`). Durch diese Datenstruktur wird sichergestellt, dass beim Erreichen der Kapazitätsgrenze der älteste Wert aus der Warteschlange entfernt wird („First-In-First-Out“). Dabei bleibt das manuelle Speicher-Management komplett erspart.

Der folgende Codeausschnitt zeigt die Kernlogik der `update`-Methode. Diese berechnet bei jedem neuen Bild den gleitenden Durchschnitt (`numpy.mean`) und schaltet anschließend den Zustand entsprechend um.

**Listing 5.2:** Zustandsermittlung mittels gleitendem Durchschnitt

```
def update(self, prey_confidence, frame_idx):
    self.confidence_history.append(prey_confidence)
    self.last_seen = frame_idx

    # Calculate moving average
    avg_confidence = np.mean(self.confidence_history)

    # Determine new state based on aggregated confidence
    if avg_confidence >= self.threshold:
        self.current_state = State.CAT_WITH_PREY
    else:
        self.current_state = State.CAT_NO_PREY
```

Die Klasse `StateMachine` verwaltet die verschiedenen `TrackState`-Objekte in einem Dictionary (`track_id` → `TrackState`). Die Methode `cleanup_stale_tracks` iteriert regelmäßig über dieses Dictionary und entfernt die Objekte, die seit mehr als 30 Einzelbildern nicht mehr vom Tracker zugeordnet wurden konnten, um Speicherlecks zu vermeiden.

Sobald eine oder mehrere Katzen mit Beute im Maul erkannt werden, wirft das `StateMachine`-Objekt ein Ereignis und meldet, dass der Eintritt von Katzen nicht erlaubt ist. Andernfalls, d. h. wenn alle Katzen sauber sind, meldet das Ereignis, dass der Eintritt erlaubt ist.

### 5.1.3 Die Orchestrierung (`processor.py`)

Im Skript `processor.py` geschieht die tatsächliche Verarbeitung der Einzelbilder. Die Klasse `CatFlapProcessor` verwaltet ein `StateMachine` Objekt, zählt wie viele Bilder

pro Sekunde verarbeitet werden können und bietet die Methode `process_frame` an. Diese bildet das Herzstück der Klasse, sie bekommt ein Videoeinzelbild als Parameter übergeben, danach wird über alle erkannten Katzen iteriert, dabei wird für jedes Objekt die Klassifikation durchgeführt. Im letzten Schritt wird dann der Zustandsautomat aktualisiert und die Visualisierungen (Begrenzungsrahmen und Zustand) werden auf das Bild eingezeichnet.

Wie schwierig es ist, qualitativ hochwertige Daten von Katzen mit Beute zu sammeln, wurde im Kapitel 3 bereits erwähnt. Um den Klassifikator gezielt zu verbessern, wurde ein Mechanismus implementiert, der Bilder speichert, bei denen das Modell unsicher war (Konfidenzwert zwischen 0,3 und 0,7). Dadurch können die Szenarien, die dem Modell am schwersten fallen, gezielt verbessert werden. Dieses Feature wird mit dem Kommandozeilenparameter `-save_uncertain` aktiviert. Um Speicher zu schonen, wird nach Auslösung des Mechanismus die Speicherung derselben `track_id` für 30 Einzelbilder gesperrt.

## 5.2 Die Verbindung zur Hardware (`catflap.py`)

Die im Skript `catflap.py` enthaltene Klasse `Catflap` stellt dabei die Hardwareumgebung zur Verfügung. Durch Polymorphismus können verschiedene Arten erzeugt werden. Dabei liefert die Methode `capture_continuous` das nächste Bild der Kamera. Außerdem bietet die Klasse Methoden zum schließen `lock_in` / `lock_out` und öffnen `unlock_in` / `unlock_out` der Klappe.

- `lock_in`: Katze darf Haus nicht betreten.
- `lock_out`: Katze darf Haus nicht verlassen.
- `unlock_in` und `unlock_out`: Heben die jeweilige Sperre auf.

Die Ableitung `SimulationCatflap` wird zur Simulation verwendet, die Videoeinzelbilder liefert hier ein Video, sperren und entsperren der Klappe wird durch eine Ausgabe auf die Konsole simuliert. Für den realen Betrieb gibt es die Klassen `ServoCatflap`, die mithilfe eines Servo-Motors sperrt, und `SureCatflap`, die als Anbindung zu einer vorhandenen SureFlap-Katzenklappe [11] verwendet werden kann.

## 5.3 Ausführungsumgebungen

Das System kann je nach Einsatzzweck in verschiedenen Umgebungen ausgeführt werden, die jeweils eigene Skripte besitzen.

### 5.3.1 Die lokale Testumgebung (`main.py`)

Das Skript `main.py` wurde entwickelt um neu trainierte Modelle schnell auf der lokalen Umgebung (z.b. Windows Laptop) validieren zu können. Dabei liest das Skript eine Videodatei ein. Danach durchläuft jedes Videoeinzelbild die `process_frame` Methode der Klasse `CatFlapProcessor`. Anschließend wird die Visualisierung auf das Video eingezeichnet und abgespielt.

### 5.3.2 Die Mikrocontroller Testumgebung (`pi_main.py`)

Um das Zusammenspiel aller Komponenten sinnvoll auf dem Endgerät testen zu können wurde das `pi_main.py` Skript entwickeltentwickelt. Dadurch ist es möglich, jeden Teil – also Detektor, Klassifikator, Servo und Kamera – isoliert zu testen. Außerdem ist dieser teil wichtig zum zu testen wie viele Bilder pro Sekunde der Raspberry Pi 5 verarbeiten kann. Ansonsten funktioniert das Skript gleich wie `main.py`.

### 5.3.3 Die Reale Welt (`pi_server.py`)

Für den realen Betrieb reichen die vorher erwähnten Skripte nicht aus. Damit man während des Betriebes die Klappe manuell sperren / entsperren und die aktuellen Kamera-Aufnahmen einsehen kann, werden im Skript `pi_main.py` mithilfe des `flask`-Paketes diese Funktionen als HTTP Endpunkte zur Verfügung gestellt.

Auch hier wird ein `CatFlapProcessor`-Objekt erzeugt und verwaltet, dieses bekommt kontinuierlich die Einzelaufnahmen der Kamera. Sollte sich der Zustand des darin enthaltenen `TrackState`-Objekts ändern, wird, wie in Kapitel 5.1.2 beschrieben, ein Ereignis geworfen. Sind alle Katzen sauber, wird die Klappe geöffnet, ansonsten wird sie gesperrt. Wurde die Katzenklappe entsperrt, so startet ein Zeitmesser, der die Klappe nach 30 Sekunden wieder schließt.

# Kapitel 6

# Tests und Analysen

In diesem Kapitel wird die Leistungsfähigkeit des gesamten Systems evaluiert. Die Beurteilung erfolgt anhand von realen Videodaten, die dem System während der Trainingsphase nicht präsentiert wurden. Im Fokus steht dabei die Frage, ob die Verbesserungen durch zeitliche Aggregation, den CLAHE-Algorithmus und das asymmetrische Zusammenschneiden zuverlässiger sind als die alleinige Verwendung der Objekterkennung und Klassifikation. Dabei ist es interessant zu sehen, in welchen Szenarien diese Zusätze das System deutlich beeinflussen.

## 6.1 Modell-Performance (Training & Validation)

Da die YOLO-Modelle einen wesentlichen Teil der Zuverlässigkeit des Systems beeinflussen, werfen wir zuerst einen Blick auf deren isolierte Leistung.

In diesem Unterkapitel wird vor allem auf die Trainingsmetriken eingegangen. Besonders Interessant sind dabei die Verlustfunktionen, welche angeben wie viele Fehler auf den jeweiligen Datensatz (Training oder Validierung) passiert sind. Die Informationen der Wahrheitsmatrizzen geben dabei Einsicht wie viele Objekte erkannt, fälschlicherweise erkannt oder vom Modell erfunden wurden.

### 6.1.1 Leistung der Objekterkennung

Die Trainingsmetriken (Abbildung 6.1) für den Katzengesicht-Detektor, zeigen einen kontinuierlich sinkenden Trainingsfehler (*engl. Loss*) und eine hohe Genauigkeit (*engl. Precision*) in der Validierung im Verlauf von 100 Epochen.

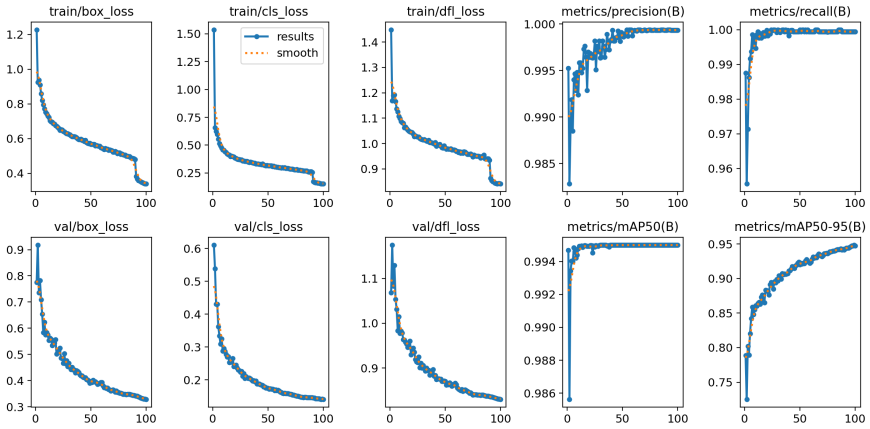



**Abbildung 6.1:** Trainingsverlauf und Metriken des YOLO-Objektdetektors.

Die Wahrheitsmatrix (*engl. Confusion Matrix*) der Objekterkennung (Abbildung 6.2) verdeutlicht die Präzision bei der Lokalisierung. Sie zeigt, dass bei der Validierung 1.676 Katzengesichter korrekt erkannt wurden. Das Modell hat jedoch zwei Katzengesichter erkannt, obwohl dort keines war, und eines hat es komplett übersehen. Die Matrix belegt also dass der Detektor nahezu fehlerfrei auf den Validierungsdaten funktionierte.

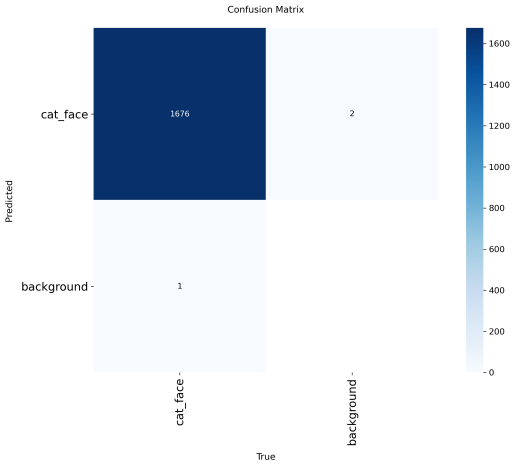



**Abbildung 6.2:** Wahrheitsmatrix (Detektor).

Eine wichtige Metrik, welche auch auf den Validierungsdaten beruht, ist die Präzision-Trefferquote-Kurve (*engl. Precision-Recall-Curve*), sie gibt das Verhältnis zwischen richtig erkannten Objekten und Fehlalarmen an wie sicher das Modell ist. In diesem Fall zeigt der Verlauf, dass das Modell extrem sicher ist und nicht raten muss, wenn es ein Katzengesicht sieht.

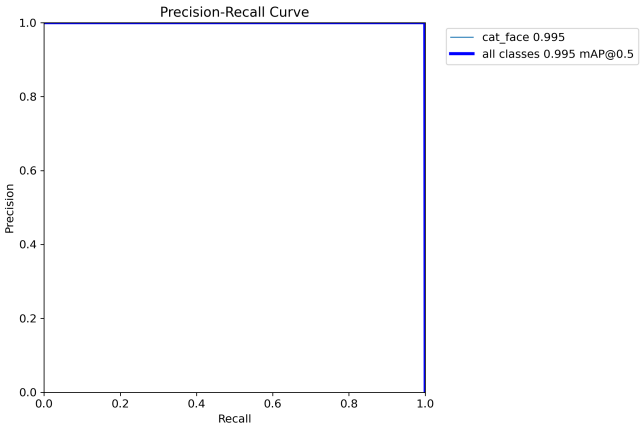



**Abbildung 6.3:** Präzision-Trefferquote-Kurve (*engl. Precision-Recall-Curve*).

Die F1-Maß-Kurve (Abbildung 6.4) zeigt wie sich die Leistung des Modells bei Änderung des Konfidenzschwellenwerts [13] verändert. Dadurch können wir den optimalen Schwellenwert für den realen Betrieb des Modells ermitteln. Da die Kurve bis $\approx 0,85$ flach bleibt, d.h. hier wird keine Katze übersehen, fällt sie $>$ 0,85 stark ab. Das zeigt, dass ab diesem Schwellenwert auch reale Katzen leicht übersehen werden. In der Praxis zeigt sich, dass bei einem Konfidenzschwellwert von $\approx 0,75$ ein gutes Verhältnis zwischen „Katze wird nicht erkannt“ und „es wird eine Katze erkannt, wo sich keine befindet“ entsteht.

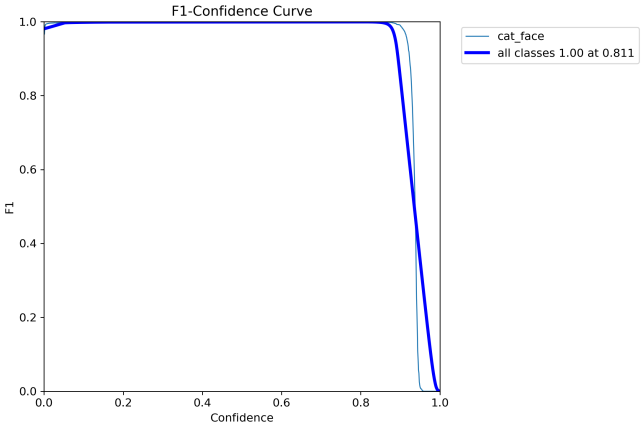



**Abbildung 6.4:** F1-Maß-Kurve.

Die Abbildung 6.5 liefert einen Eindruck auf die Erkennungsleistung des Modells, wenn man es auf die Validierungsdaten ansetzt.

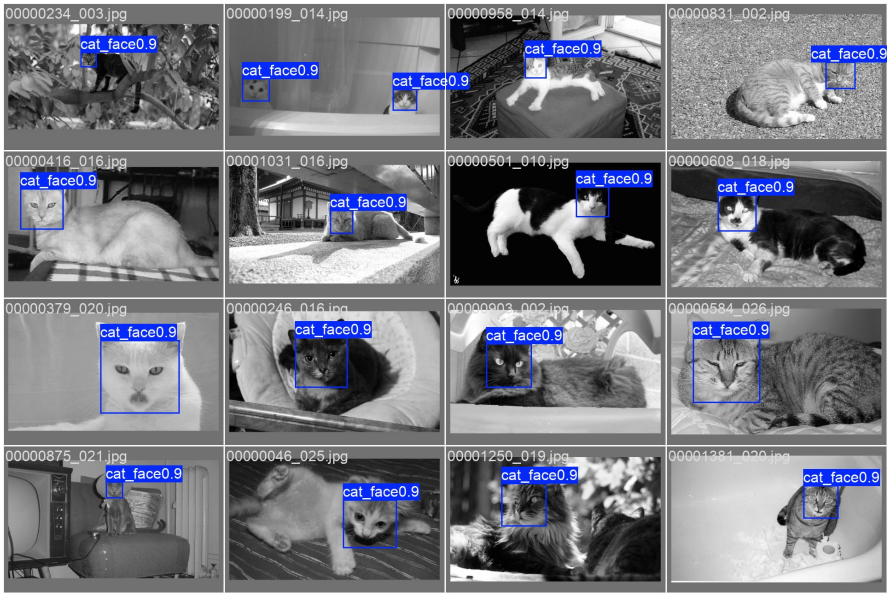



**Abbildung 6.5:** Vorhersagen des Detektors auf dem Validierungsdatensatz.

### 6.1.2 Leistung der Bildklassifikation

Die folgenden Trainingsmetriken des Beute-Klassifikators zeigen, dass das Modell seine Aufgabe erfolgreich gelernt hat. Beim betrachten der Verlustfunktionen wird ersichtlich, dass das Modell über die Epochen weniger Fehler sowohl bei Training als auch Validierung macht, daraus können wir schließen, dass der Klassifikator lernt ohne sich dabei zu viel auf den Trainingsdatensatz anpasst.

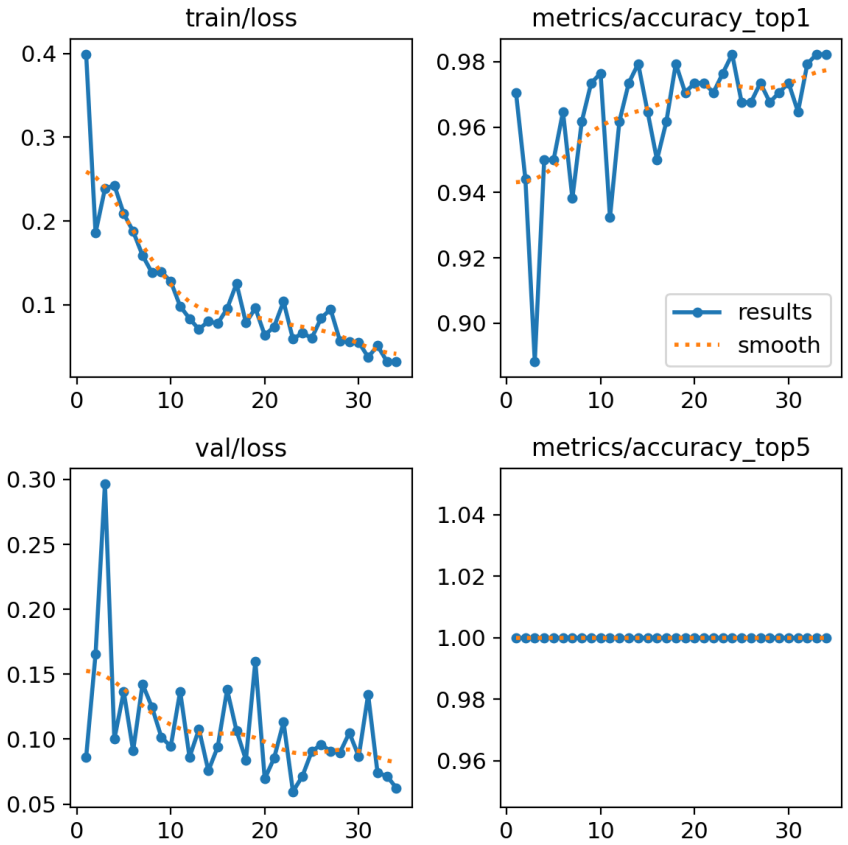



**Abbildung 6.6:** Trainingsverlauf und Metriken des YOLO-Klassifikators.

Wenn man einen Blick auf die Wahrheitsmatrix (Abbildung 6.7) wirft, zeigt sich, dass das Modell auf den Validierungsdatensatz großartig funktionierte. Bei den insgesamt 335 Fotos, gab es nur 5 Fehler bei der binären Entscheidung „Beute“ und „Keine Beute“.

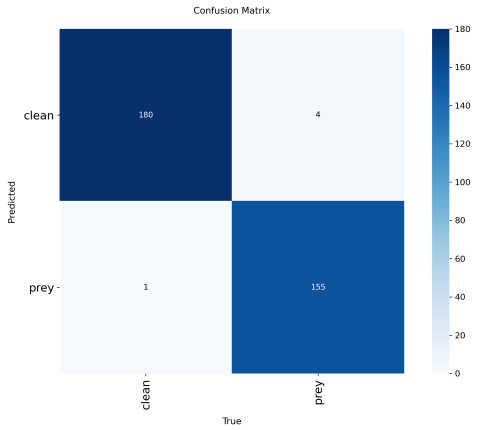



**Abbildung 6.7:** Confusion Matrix des Klassifikators auf dem Validierungsdatensatz.

Mithilfe der Abbildung 6.8 lässt sich gut erkennen, wie gut das Modell mit unbekannten Daten umgeht.




**Abbildung 6.8:** Beispielhafte Vorhersagen des Klassifikators auf dem Validierungsdatensatz.

Rein auf dem Papier würden diese Modelle fast fehlerfreie Aussagen treffen. Die entscheidende Frage ist jedoch, wie verhält sich das System im Praxiseinsatz und welche Vorteile bietet die Verarbeitungskette gegenüber der reinen Detektion und Klassifikation.

## 6.2 Evaluationsmethodik und Datensatz

Die Testbasis für das System bilden insgesamt 7 Videosequenzen, die sich auf zwei Kategorien aufteilen. Die Dateinamen der Kategorien geben Aufschluss wie viele Katzen und in welcher Farbe in dem Video vorkommen.

- **Positive Samples (Mit Beute):** 4 Videos (`black.mp4`, `orange.mp4`, `orange2.mp4`, `two_cats_black_white.mp4`)
- **Negative Samples (Ohne Beute):** 3 Videos (`black.mp4`, `orange.mp4`, `two_cats_grey_black.mp4`)

Die Evaluation fand automatisiert über das Testskript `main.py` statt. Dabei durchläuft jedes Einzelbild des Videos die gesamte Verarbeitungskette. Ein Video wird positiv klassifiziert, wenn die Katze Beute trägt. Andernfalls, sprich die Katze ist sauber, wird es negativ klassifiziert.

### 6.2.1 Definition der System-Metriken

Nachdem alle Videosequenzen die Verarbeitungskette durchlaufen sind, kann man die Metriken für das gesamte System erstellen. Diese leiten sich aus Wahrheitsmatrix ab:

- **Richtig-Positiv-Ergebnis (*engl. True Positive*) (TP):** Katze mit Beute darf nicht ins Haus (korrekte Sperrung).
- **Falsch-Negativ-Ergebnis (*engl. False Negative*) (FN):** Katze mit Beute darf ins Haus (Beute gelangt fälschlicherweise hinein).
- **Richtig-Negativ-Ergebnis (*engl. True Negative*) (TN):** Katze ohne Beute darf ins Haus (korrekter Durchlass).
- **Falsch-Positiv-Ergebnis (*engl. False Positive*) (FP):** Katze ohne Beute darf nicht ins Haus (Fehlalarm).

Im Kontext einer Katzenklappe besitzt die Katze aus Nutzersicht die höchste Priorität. Ein Falsch-Positiv-Ergebnis (Fehlalarm) bedeutet, dass das Haustier grundlos vor einer verschlossenen Tür steht. Um das Vertrauen des Tieres in die Klappe nicht zu zerstören, liegt der Fokus der Evaluierung primär auf der Vermeidung solcher Fehlalarme (hohe Präzision). Ein seltenes Falsch-Negativ-Ergebnis (*engl. False Negative*) (eine Maus gelangt ins Haus) wird im Tausch für eine stressfreie Nutzung der Klappe durch die Katze bewusst in Kauf genommen.

## 6.3 Ergebnisse der Evaluation

Das System verarbeitete die Videosequenzen mit einer durchschnittlichen Inferenzgeschwindigkeit von knapp 8 FPS. Da der Zustandsautomat eine Historie von 30 Einzelbildern (`history_length=30`) auswertet, entspricht dies einer zeitlichen Glättung von knapp vier Sekunden. Bei den folgenden Metriken ist zu beachten, dass ein Video als korrekt eingestuft wird, wenn das System grundsätzlich richtig entscheidet. Kleine Schwankungen werden dabei nicht berücksichtigt. Die Erkennungsleistung über den Testdatensatz ist in der folgenden Wahrheitsmatrix dargestellt.

| | **Vorhersage: Beute** | **Vorhersage: Keine Beute** |
|---|---|---|
| **Realität: Beute** | 4 (TP) | 0 (FN) |
| **Realität: Keine Beute** | 0 (FP) | 3 (TN) |

**Tabelle 6.1:** Wahrheitsmatrix der Systemevaluation auf dem Videodatensatz (Schwellenwert = 0.8).

Aus diesen Werten ergeben sich die Leistungsmetriken des Systems:

- **Präzision (*engl. Precision*):** $\frac{TP}{TP+FP} = \frac{4}{4+0} = 100,0\,\%$
- **Trefferquote (*engl. Recall*):** $\frac{TP}{TP+FN} = \frac{4}{4+0} = 100,0\,\%$

### 6.3.1 Analyse der Systemleistung und Kompromisse

Die Ergebnisse der Leistungsmetriken würden bedeuten, dass das System perfekt funktioniert. Während uns diese Metriken lediglich mitteilen, dass das System für das gesam-

te Video die richtige Entscheidung getroffen hat, ist für die Zuverlässigkeit die Stärke des Flackerns viel interessanter. Um ein tieferes Verständnis für die Leistung zu erlangen, verwenden die nachfolgenden Ablationsstudien eine andere Metrik: die Anzahl der Zustandswechsel pro 100 Bilder. Dadurch können wir die Robustheit des Systems wirklich bewerten.

## 6.4 Ablationsstudie: Relevanz des Zustandsautomaten

Um die Wirksamkeit und absolute Notwendigkeit der zeitlichen Aggregation (Zustandsautomat) zu validieren, wurde eine Ablationsstudie durchgeführt. Achtung! Einige Zustandswechsel sind gewollt: Auch wenn die Katze keine Beute trägt, wird ihr der Zugang so lange verweigert, bis sie beweist, dass sie sauber ist. Bei dieser Studie wurde das exakt selbe System zweimal auf eine Auswahl von Testvideos angewendet:

1. **Volles System:** Mit aktivem Zustandsautomaten und einem „gleitenden Durchschnitt“ (*engl. moving average*) über 15 Einzelbilder (`history_length=15`).
2. **Ablation:** Da der Zustandsautomat deaktiviert wurde, basierte die Entscheidung über die Klappensperre ausschließlich auf dem Ergebnis des aktuellen Einzelbildes. (`history_length=1`).

Die Ergebnisse (siehe Abbildung 6.9 und 6.10) zeigen drastische Unterschiede. Egal in welchen Szenario das Volle System liefert immer die bessere Leistung. Die Klappe wird durch den Zustandsautomat nicht dauernd geöffnet und geschlossen, er beruhigt die Fehlerrate und gleicht die Ungenauigkeiten des Detektors und Klassifikators aus.

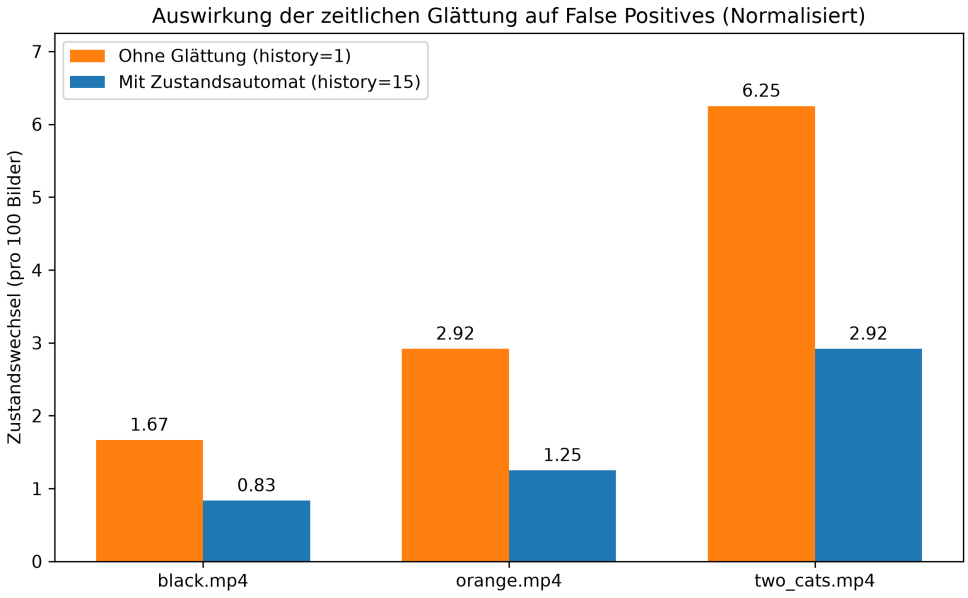



**Abbildung 6.9:** Zustandswechsel bei Videos ohne Beute.

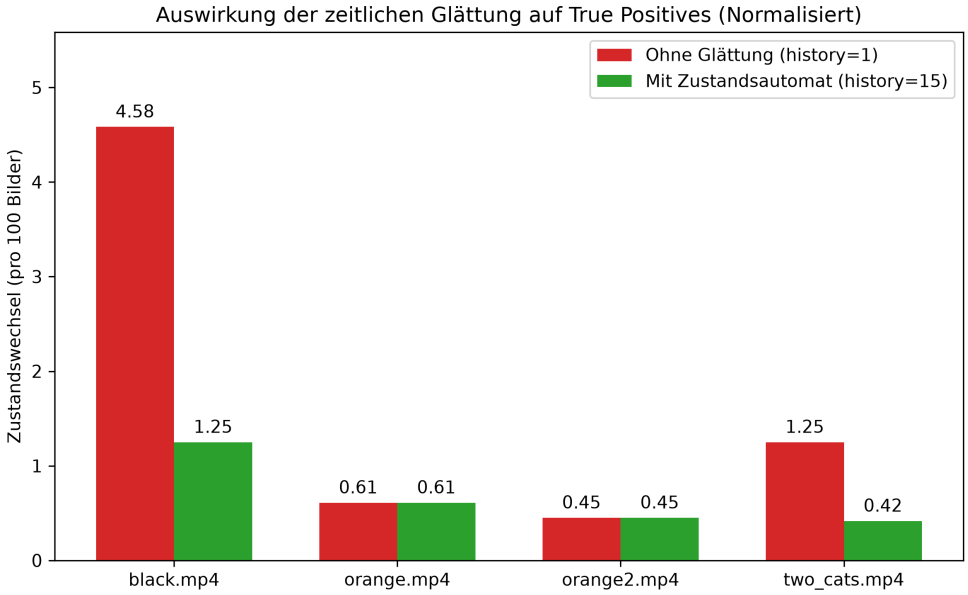



**Abbildung 6.10:** Zustandswechsel bei Videos mit Beute.

### 6.4.1 Ablationsstudie: Relevanz der Randerweiterung

Um isoliert beurteilen zu können wie wichtig diese Funktionalität für unser System wirklich ist, wird der Zustandsautomat völlig deaktiviert (`history_length=1`).

Hier wird ganz schnell klar, ohne die Randerweiterung fängt das System an zu raten. Dadurch das nicht die vollständige Beute im Bild ist, kann das Modell keine Entscheidung treffen, egal ob mit oder ohne Beute, die Vorhersagen sind unsicher und ungenau.

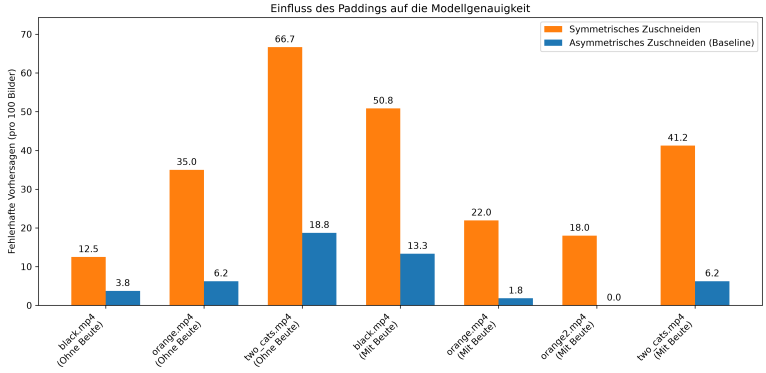



**Abbildung 6.11:** Isolierte Auswirkung des asymmetrischen Croppings auf die rohe Systemstabilität (ohne Zustandsautomat).

### 6.4.2 Ablationsstudie: Kontrastverstärkung (CLAHE) im Klassifikator

Für diese Studie wurden zwei unterschiedliche Klassifikatoren verwendet. Einer wurde mit Kontrastverstärkten (CLAHE) und der andere mit normalen Graustufen Bildern trainiert. Um den Effekt dieser Funktionalität realistisch bewerten zu können, wurden die Testvideos in Graustufen konvertiert. Die Ergebnisse (siehe Abbildung 6.12) zeigen, dass der CLAHE-Filter einen interessanten Kompromiss mit sich bringt.

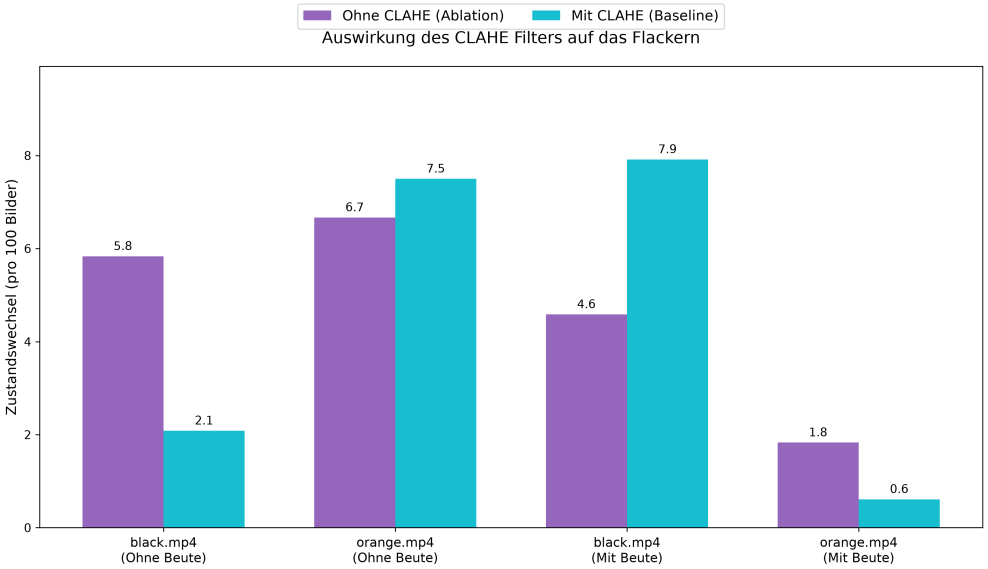



**Abbildung 6.12:** Vergleich der normalisierten Zustandswechsel bei Videos *mit Beute* mit und ohne CLAHE-Filter im Klassifikator auf Graustufenbildern.

Während der CLAHE-Filter das Flackern bei der schwarzen Katze ohne Beute (`black.mp4`) stark reduziert, ist es bei dem Video mit Beute fast doppelt so stark. Bei den Videos mit den Orangen Katzen (`orange.mp4`) halten sich die Zustandswechsel pro 100 Bilder egal ob mit oder ohne Beute im Rahmen und sind daher für unsere Studie nicht ausschlaggebend

Da aus unserer Sicht wichtiger ist, dass die Katze nicht vor verschlossener Tür steht obwohl sie sauber ist, bringt der CLAHE-Filter genau in diesem Punkt, bei Katzen mit dunklerem Fell) einen klaren Vorteil mit sich.

### 6.4.3 Ablationsstudie: Einstufige vs zweistufige Klassifikation

Zur Validierung der Architektur unserer zweistufigen Kette (Detektor + Klassifikator) wurde für diese Studie ein einstufiger Objektdetektor (YOLO26) trainiert. Dieser besitzt zwei Klassen: „Katze mit Beute" und „Katze ohne Beute". Beide Klassen besitzen gleich viele Daten.

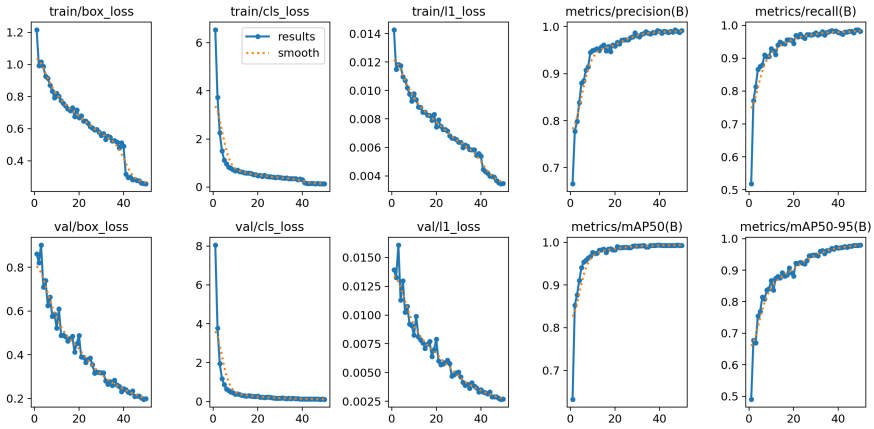



**Abbildung 6.13:** Trainingsmetriken des einstufigen Objektdetektors.

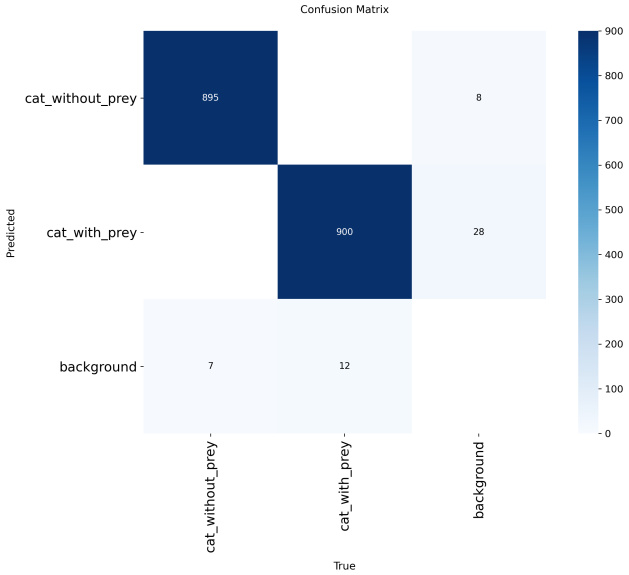



**Abbildung 6.14:** Konfusionsmatrix des einstufigen Objektdetektors auf den Validierungsdaten.

Um die Modelle isoliert zu vergleichen, wurde der Zustandsautomat wieder deaktiviert (`history_length=1`). Dabei wurde evaluiert wie viele Fehlklassifikationen beide Systeme trafen, sowie wie viele Katzen insgesamt im Video übersehen wurden.

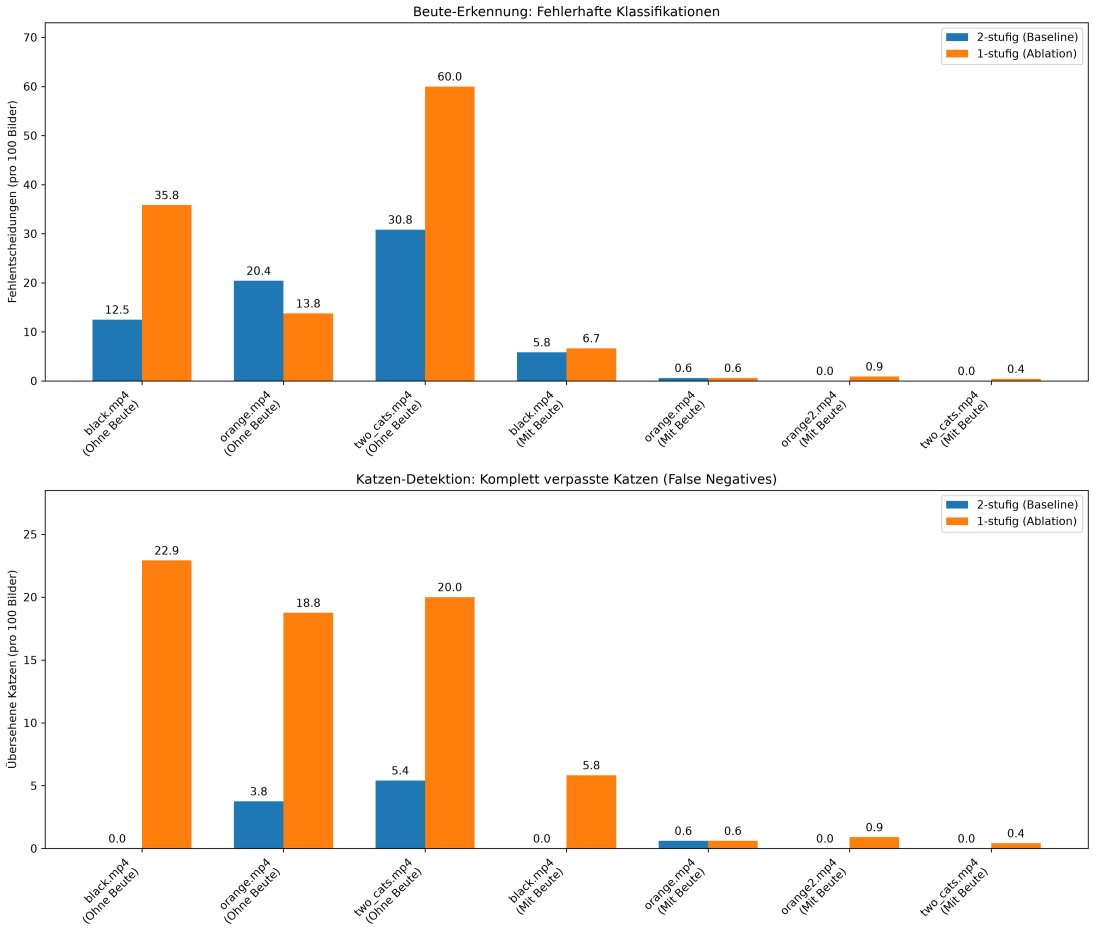



**Abbildung 6.15:** Vergleich: Klassifikationsfehler (oben) und komplett übersehene Katzen (unten) pro 100 Frames.

Die Ergebnisse (siehe Abbildung 6.15) verdeutlichen, dass der zweistufige Ansatz in den meisten Szenarien die bessere Wahl ist.

# Kapitel 7

# Diskussion

Die in Kapitel 6 präsentierten Testergebnisse liefern ein klares Bild der Systemleistung. In diesem Kapitel werden diese Ergebnisse interpretiert und kritisch bewertet. Dabei wird untersucht, inwiefern die Forschungsziele erreicht wurde, wo die grenzen des Systems liegen und welche Funktionalitäten für die Qualität ausschlaggebend waren.

## 7.1 Beantwortung der zentralen Forschungsfrage

Die primäre Forschungsfrage dieser Arbeit war, ob eine Verarbeitungskette aus Objekterkennung, Bildklassifikation, Objektverfolgung und zeitlicher Zustandsmodellierung, eine robustere Beute-Erkennung ermöglicht gegenüber Einzelbildmethoden.

Die Evaluierung liefert hierauf eine eindeutig Antwort. Ein einzelbildbasierter Ansatz ist bei der Analyse von Videostreams stark anfällig für Fehler. Bewegungsunschärfe (Motion Blur), wenn die Katze ins Bild läuft, partielle Verdeckungen (Okklusionen) der Beute durch die Pfoten oder rasche Änderungen der Beleuchtung führen zu starken Ungenauigkeiten der Klassifikation. Ohne die zeitliche Aggregation im Zustandsautomaten würde die Zugangskontrolle ständig zwischen den Zuständen „verriegelt“ und „entriegelt“ wechseln (Flackern).

Durch die hilfe der `StateMachine` in Kombination mit ByteTrack wird diese Problem elegant gelöst. Die zeitliche Aggregation über eine Historie von Einzelbildern erzeugt ein robustes Signal, das Ausreißer effektiv glättet. Die Ablationsstudie „Relevanz des Zustandsautomaten“ zeigt, dass das System selbst dann richtig entscheidet, wenn einzelne Bilder falsch klassifiziert werden. Somit ist der Beweis erbracht, dass der Zustandsautomat in diesem Anwendungsfall ein wesentlicher Teil ist und zwingend für die Zuverlässigkeit im realen Betrieb ist.

## 7.2 Bewertung des Architektur-Ansatzes

Die Entscheidung, das Problem in zwei separate neuronale Netze (YOLO-Detektor für das Gesicht und YOLO-Klassifikator für den Zustand) zu unterteilen, hat sich als äußerst vorteilhaft erwiesen.

### 7.2.1 Stärken der Verarbeitungsketten-Kaskadierung

Der übliche Ansatz im Bereich maschinelles Sehen bestünde darin, ein einziges Objekterkennungsmodell darauf zu trainieren, direkt die Klassen „Katze“ und „Beutetier“ im Gesamtbild zu finden. Bei einer Katzenklappe scheitert dieser Ansatz jedoch häufig. Eine Maus im Maul einer Katze nimmt im Gesamtbild der Kamera nur einen verschwindend geringen Bruchteil der Pixel ein und ist durch das Maul und die Schnurrhaare stark verdeckt. Standard-Objektdetektoren haben immense Schwierigkeiten, derart kleine, okkludierte Objekte zuverlässig mit einem Begrenzungsrahmen zu versehen.

Der in dieser Arbeit gewählte Kaskadierungs-Ansatz umgeht dieses Problem elegant. Der erste Detektor muss lediglich das vergleichsweise große und markante Katzengesicht finden – eine triviale Aufgabe für moderne Architekturen wie YOLOv11, was sich in der hohen Erkennungsstabilität niederschlug. Durch das anschließende Herausschneiden (Zuschneiden) dieses Bereichs wird die Aufmerksamkeit des nachgelagerten Klassifikators exklusiv auf die relevante Region fokussiert (Aufmerksamkeitsmechanismus *by design*). Der Klassifikator muss keine kleine Maus im Hintergrund des Gartens suchen, sondern analysiert lediglich die Pixelstruktur direkt am Maul der Katze.

### 7.2.2 Die Rolle der asymmetrischen Randerweiterung

Dadurch das die Annotationen des Datensatzes im Detektor („Cat Dataset“ von Crawford [4]) den unteren Teil des Mauls komplett verdecken, ist es unerlässlich für Training und Inferrenz den Begrenzungsrahmen nach untenhin Proportional zu erweitern. Die zugehörige Ablationsstudie 6.7 zeigt hier ganz genau, wie Fehleranfällig das System ohne dieser Funktionalität wäre.

## 7.3 Schwächen, Limitationen und Fehlerquellen

Aus der Analyse werden die Limitationen des aktuellen Systems genau aufgezeigt. Dadurch entsteht das Potenzial für zukünftige Verbesserungen:

1. **Verlust von Tiefeninformationen:** Dadurch das die Kamera ein 2D Bild aufnimmt kann nicht unterschieden werden, wie weit weg sich ein Objekt unterhalb des Katzenmauls befindet. Daraus folgt, dass das Modell den Unterschied zwischen Hintergrund und Beute nicht zuverlässig trennen kann.
2. **Extreme Bewegungsgeschwindigkeit:** Wenn eine Katze fluchtartig durch die Klappe rennt, bleibt sehr wenig Zeit eine klare Aussage zu treffen. Bei einer Laufzeit von ca. 22 FPS verbleiben dem System nur eine Handvoll Einzelbilder zur Analyse. In diesem Szenario kann die `StateMachine` ihre Stärke (die zeitliche Glättung) nicht voll ausspielen und die Katze muss warten bis der Zustandsautomat eine klare Entscheidung trifft.
3. **Farbe der Katze:** Wenn eine Katze ein sehr dunkles Fell besitzt, hat die Klassifikation Schwierigkeiten und verliert an Genauigkeit. Hier ist es essentiell Daten aus dem Praxisbetrieb zu sammeln, bei denen sich das Modell unsicher war, um genau diese Fälle besser trainieren zu können.

## 7.4 Lokale Datenverarbeitung

Ziel dieser Arbeit war es auch, dass das System ressourcenschonend konzipiert wird um sicherzustellen dass die Inferrenz auch auf einem Mikrocontroller lokal operieren kann. Daher viel die Entscheidung auf eine leichtgewichtige Architektur (YOLO26n) und optimierte Formate (TensorFlow Lite). Die dadurch verlorene Genauigkeit wird durch die erreichte Leistung gerechtfertigt (ca. 15 FPS).

Dieser Ansatz beweist, dass eine komplexe Aufgabe auf ressourcenbegrenzter Hardware durch eine intelligent gewählte Architektur gelöst werden kann, ohne dass die Robustheit zu stark beeinträchtigt wird. Dadurch wird eine Auslagerung von Berechnungen in externe Rechenzentren vermieden. Das spart Geld und macht die Lösung zuverlässiger, da keine Internetverbindung notwendig ist.

Abschließend lässt sich sagen, dass die Architektur ihre Aufgabe größtenteils erfüllt. Die verbleibenden Fehlalarme sind kein Architekturfehler, sondern zeigen die Grenzen des begrenzten Klassifikationsdatensatzes auf. Mithilfe des in Kapitel 5 integrierten Modells zum Speichern von Bildern, bei denen das Modell unsicher war, können genau diese Lücken im Datensatz ergänzt werden. So kann das Modell kontinuierlich verbessert werden.

# Kapitel 8

# Zusammenfassung und Ausblick

In der vorliegenden Bachelorarbeit wird ein robustes Zugangssystem für Katzen entwickelt, dass das eindringen von Beutetieren (z. B. Mäuse oder Vögel) in den Wohnraum verhindert. Durch die Kombination von mehrschitigen Lernen (eng. Deep Learning) und zeitlicher Aggregation wurde ein System geschaffen, das die einzelbildbasierte Objekterkennung bei Weitem übertrifft.

## 8.1 Zusammenfassung der Erkenntnisse

Die theoretischen Annahmen konnten durch die praktische Evaluierung erfolgreich validiert werden. Ein klassischer In diesem Anwendungsfall ist der monolithische Objektdetektor bei weitem ungenauer als die in der Arbeit vorgestellte Verarbeitungskette, die aus einem Objektdetektor, einer Bildklassifikation und einer zeitlichen Aggregation besteht.

Zunächst detektiert ein YOLO-Detektor das Gesicht der Katze. Um zu verhindern, dass die eventuelle Beute im Maul der Katze okkludiert wird, wird der Ausschnitt des Gesichts um 35 Prozent nach unten erweitert. Im zweiten Schritt analysiert ein YOLO-Klassifikator, ob sich Beute im Maul der Katze befindet. Um Fehlklassifikationen auszugleichen, muss eine Katze beweisen, dass sie sauber ist. Erst dann darf sie in den Wohnraum eintreten. Dazu wird die Katze über mehrere Sekunden hinweg mithilfe von ByteTrack verfolgt. Der gleitende Durchschnitt der letzten 30 Einzelbilder (was etwa zwei Sekunden entspricht) entscheidet dann, ob die Katze ins Haus darf oder nicht.

Die Evaluierung mit realen Videodaten zeigte den Erfolg des gewählten Architektur-Ansatzes. Das System konnte alle Testdaten erfolgreich und korrekt auswerten und so feststellen, ob die Katze sauber ist oder nicht. Der gewählte Schwellenwert von 0,8 spiegelt dabei die Priorität wider, dass Katzen im Falle von Systemunsicherheiten Zugang zum Haus haben. Dadurch wird sichergestellt, dass keine Katze zu Unrecht ausgesperrt wird. Das System erweist sich somit als sehr praxistauglich und läuft mit durchschnittlich 15 Bildern pro Sekunde ressourcenschonend auf lokalen Endgeräten.

Somit konnte die zentrale Forschungsfrage bejaht werden. Eine Kombination aus Objekterkennung, Objektverfolgung und zeitlicher Glättung ist deutlich robuster als herkömmliche Einzelbildauswertungen.

## 8.2 Ausblick und Weiterentwicklungsmöglichkeiten

Obwohl das System in seiner aktuellen Form bereits zuverlässig Beutetiere aussperrt, zeigt die Praxis, dass auch dieser Ansatz Schwächen und Limitierungen hat. Das eröffnet die Möglichkeit, konkrete Ansatzpunkte für zukünftige Verbesserungen zu identifizieren.

### 8.2.1 Kontinuierliches Sammeln schwieriger Negativbeispiele

Die vielversprechendste Methode, um die Systemleistung zu steigern, ist die Erweiterung des Datensatzes für Katzen mit Beute. Um dieses Ziel zu automatisieren, verfügt die Applikation über eine Funktionalität, die Bilder speichert, bei denen die Bildklassifikation unsicher war. Dadurch können zukünftige Arbeiten diesen stetig wachsenden Datensatz nutzen, um den YOLO-Klassifikator iterativ nachzutrainieren. Sobald der Klassifikator aus diesen Fehlern lernen kann, wird die Fehlalarmrate des gesamten Systems erheblich sinken.

### 8.2.2 Verlust der Tiefeninformationen

Eine weitere Einschränkung ist der Verlust von 3D-Informationen im Bild. Dadurch kann es passieren, dass Objekte im Hintergrund oder sogar der Körper der Katze fälschlicherweise als „im Maul der Katze“ interpretiert werden. Eine mögliche Erweiterung bestünde darin, diese 3D-Informationen mithilfe von Stereo-Kameras oder Time-of-Flight-Sensoren zurückzugewinnen. Dadurch könnten Fehlalarme herausgefiltert werden, bei denen sich das vermeintliche Beutetier weit hinter dem Maul der Katze befindet.

### 8.2.3 Hardware-Beschleunigung

Derzeit wird das System ausschließlich auf der CPU ausgeführt. Ein Wechsel auf eine dedizierte Hardware (z. B. Google Coral) würde die Anzahl der pro Sekunde verarbeiteten Bilder deutlich erhöhen. Dadurch können wir in der gleichen Zeit mehr Daten der Katze sammeln, was die zeitliche Glättung verbessern würde. Das Flackern des Systems würde dadurch noch weiter eingeschränkt.

Abschließend lässt sich zusammenfassen, dass die in dieser Arbeit entwickelte Verarbeitungskette ein äußerst kompaktes und robustes Fundament bildet. Sie löst das Problem des unerwünschten Beuteeintrags durch Katzen zuverlässig und ermöglicht zugleich spannende Erweiterungen und Verbesserungen.

# Quellenverzeichnis

## Literatur


[1] Martín Abadi u. a. „Tensorflow: A system for large-scale machine learning“. In: *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 2016, S. 265–283 (siehe S. 6).

[2] Arm Limited. *Arm Architecture*. Verfügbar unter: https://www.arm.com/architecture. 2026 (siehe S. 6).

[3] Gary Bradski. „The OpenCV Library“. *Dr. Dobb's Journal of Software Tools* (2000) (siehe S. 6).

[4] Crawford. *Cat Dataset*. Verfügbar unter: https://www.kaggle.com/datasets/crawford/cat-dataset. 2024 (siehe S. 5, 8, 29).

[5] Gemini Team u. a. „Gemini: a family of highly capable multimodal models“. *arXiv preprint arXiv:2312.11805* (2023) (siehe S. 8).

[6] Ian Goodfellow, Yoshua Bengio und Aaron Courville. *Deep Learning*. MIT Press, 2016 (siehe S. 8).

[7] Licheng Jiao u. a. „New Generation Deep Learning for Video Object Detection: A Survey“. *IEEE Transactions on Neural Networks and Learning Systems* (2022) (siehe S. 1).

[8] Glenn Jocher und Jing Qiu. *Ultralytics YOLO11*. Version 11.0.0. Verfügbar unter: https://github.com/ultralytics/ultralytics. 2024 (siehe S. 1, 2).

[9] Rudolph Emil Kalman. „A new approach to linear filtering and prediction problems“. *Journal of basic Engineering* 82.1 (1960), S. 35–45 (siehe S. 3).

[10] Yann LeCun u. a. „Gradient-based learning applied to document recognition“. *Proceedings of the IEEE* 86.11 (1998), S. 2278–2324 (siehe S. 2, 3).

[11] Sure Petcare. *Microchip Cat Flap*. Verfügbar unter: https://www.surepetcare.com/en-gb/pet-doors/microchip-cat-flap. 2026 (siehe S. 13).

[12] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer Nature, 2022 (siehe S. 2).

[13] Ultralytics. *Glossary: Confidence*. Verfügbar unter: https://www.ultralytics.com/de/glossary/confidence. 2026 (siehe S. 2, 17).

[14] Ultralytics. *YOLO Data Augmentation*. Verfügbar unter: https://docs.ultralytics.com/guides/yolo-data-augmentation. 2024 (siehe S. 8).

[15] Guido Van Rossum und Fred L Drake Jr. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995 (siehe S. 6).

[16] Yifu Zhang u. a. „ByteTrack: Multi-Object Tracking by Associating Every Detection Box“. *Proceedings of the European Conference on Computer Vision (ECCV)* (2022) (siehe S. 1, 3).

[17] Karel Zuiderveld. „Contrast limited adaptive histogram equalization“. In: *Graphics gems IV*. Academic Press Professional, Inc., 1994, S. 474–485 (siehe S. 8).